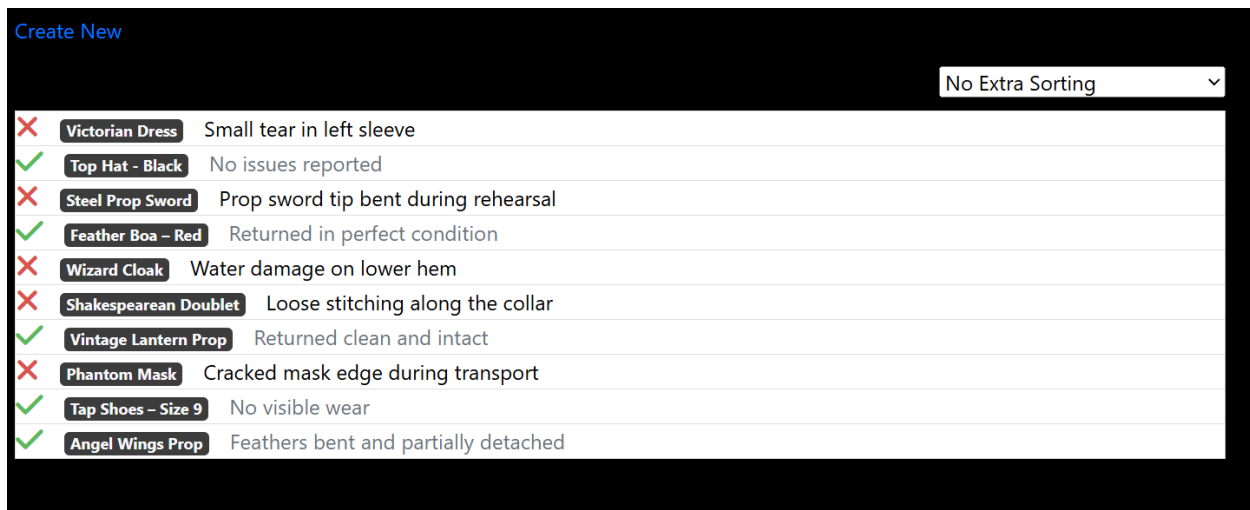


The features that I built for the TheatreCMS application are centered around delivering a complete **Rental History CRUD system** with a customized UI and enhanced client-side functionality.

1. Full CRUD Functionality

I implemented complete **Create, Read, Update, and Delete** operations for the RentalHistory entity, including validation, redirects, and error handling within the controller.

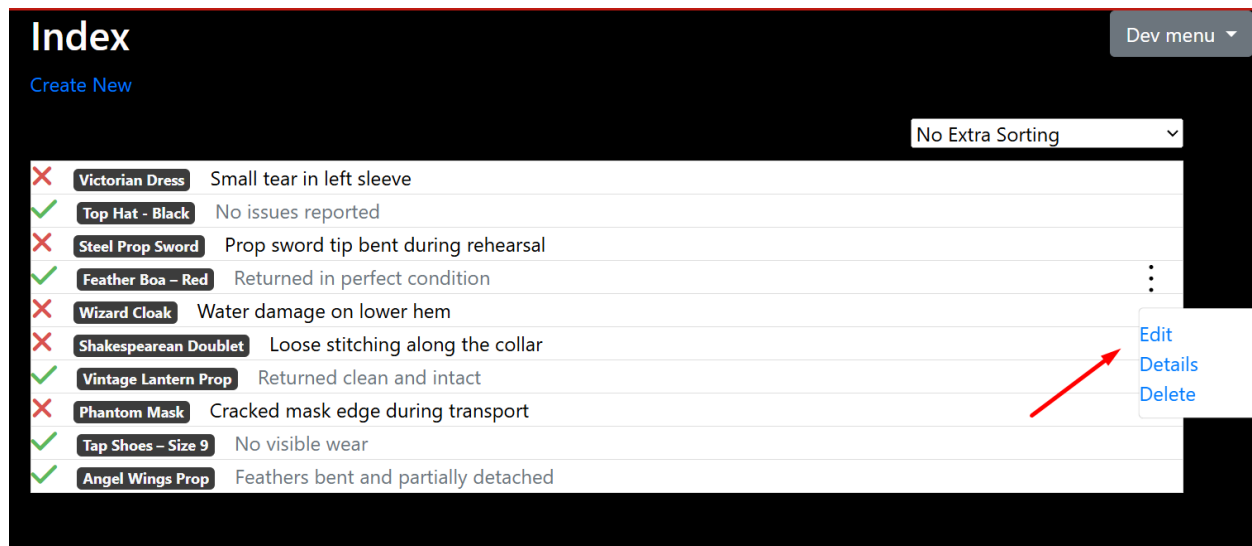


2. Custom-Styled UI for the Index View

I redesigned the default scaffolded table into an interactive layout using Bootstrap and Font Awesome:

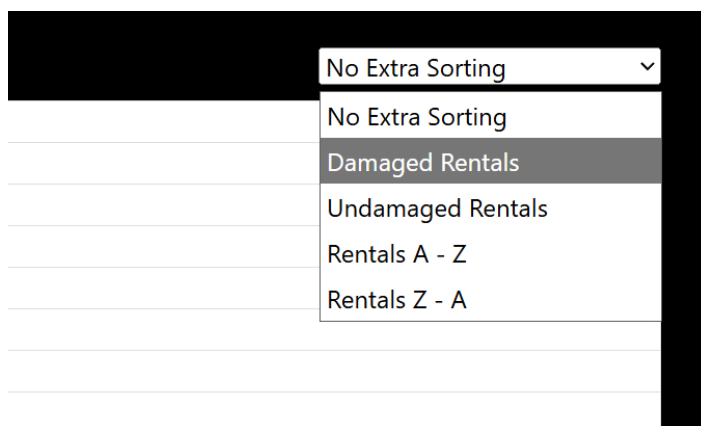
- **Bootstrap “list-group layout”** instead of a table
- **Font Awesome icons**
 - Green check for undamaged rentals
 - Red X for damaged rentals
- **Badge styling** for rental names
- **Damage text logic**
 - Gray text when no damage is present
- **Hover-activated ellipsis menu** for Edit / Details / Delete
- **Absolute-positioned action menu** for clean alignment
- **Flexbox row structure** for responsive layout

The ellipsis menu appears only when hovering a row.



3. Client-Side Sorting (No Page Reload)

I added a **sorting dropdown** with JavaScript that reorders DOM elements instantly:



This provides fast sorting without server calls.

4. Data Attributes for Sorting

Each rental row includes custom HTML data attributes used by the sorting logic:

- data-id
- data-name
- data-damaged

These attributes enable efficient client-side sorting without modifying the backend.

5. Updated Layout Integration

The view was integrated with:

- **Custom Rent Area layout**
- Font Awesome 6
- Google Fonts

This ensures consistent styling across the Rent area of the application.

6. Database Integration

I added full backend support for the Rental History feature:

- **RentalHistory model**
- **DbSet in ApplicationDbContext**
- **Entity Framework migration** to create the RentalHistories table
-

7. Cleaned-Up Controller

The controller now includes:

- Full CRUD actions
- Model validation
- Redirect logic
- Error handling for null or invalid IDs

How everything connects:

1. **User clicks a link → Routing sends request to the controller**
2. **Controller action runs → Executes C# logic**
3. **Controller uses the model → Fetches or updates data**
4. **Controller returns a view → Passes a C# model object**
5. **View renders HTML → Razor + C#**
6. **User interacts with UI → Sorting, editing, deleting**
7. **POST actions update the database → EF saves changes**

This is the full request/response cycle.

TheatreCMS3

Application Architecture

TheatreCMS3 is a **full ASP.NET MVC 5 web application** built using the **Model–View–Controller architecture**, **Entity Framework**, and **ASP.NET Identity**. The application is divided into logical sections called **Areas**, which allow different features of the site to be organized into modular components. Each Area contains its own Models, Views, and Controllers, making the project scalable and easy to maintain.

TheatreCMS3 is built using the **ASP.NET MVC 5 framework**, which organizes the entire application into three core components: **Models**, **Views**, and **Controllers**. These components work together to handle user requests, process data, and generate dynamic web pages.

When the application starts, the **Global.asax** file initializes the system by registering all Areas, routing rules, global filters, and resource bundles. This setup defines how the application responds to incoming requests and how different parts of the site are connected.

Entity Framework & Database

Entity Framework handles all communication with the SQL database. It creates tables based on Models, manages schema changes through migrations, and provides a clean way to query and update data using C# instead of raw SQL.

Authentication & Identity

The project uses **ASP.NET Identity** for user authentication and authorization. This system manages user accounts, passwords, roles, and login sessions. The Startup.cs file configures the OWIN middleware that powers the authentication pipeline.

Bundling & Minification

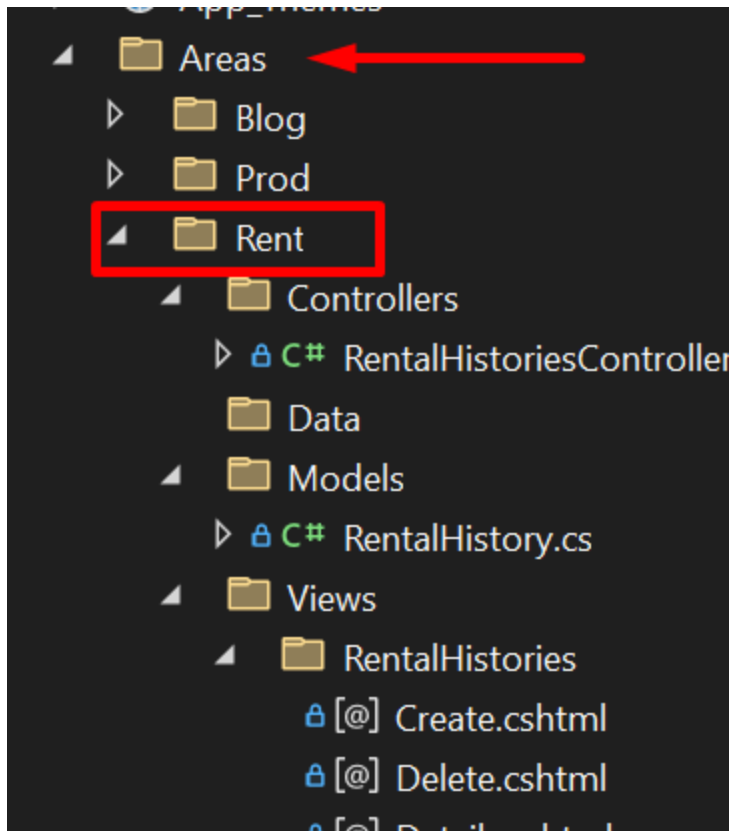
The BundleConfig file optimizes CSS and JavaScript by combining and compressing files. This improves site performance and reduces load times.

The core of the application is built around **Controllers**, which receive incoming HTTP requests, process data using **Models**, and return **Razor Views** that generate dynamic HTML. Data is stored in a SQL database and accessed through **Entity Framework**, which manages database tables, relationships, and migrations. Authentication and user management are handled through **ASP.NET Identity**, allowing the site to support login, roles, and secure access.

The project functions as a structured, modular web application where each feature is isolated into its own Area, the database is managed through Entity Framework, and the user interface is rendered through Razor Views. The MVC pattern ensures a clean separation of concerns, making the application easier to extend and maintain.

MVC Applications are divided into modular sections called **Areas**. Each Area contains its own Models, Views, and Controllers, allowing different features of the site to be isolated and organized. This makes the project scalable and easier to maintain.

The portion of code I wrote is located in the Rent Area. So my contribution was built as a self-contained MVC module inside the Rent Area, with its own Models, Views, and Controllers that integrate into the larger TheatreCMS3 application.



Program Flow

Because the TheatreCMS3 app uses the MVC pattern, the rental history feature flow is as follows:

First a user clicks a link or enters a URL. Routing sends the request to the rent controller.

Routing

Routing determines how URLs map to controllers and actions. The RouteConfig file defines global routes, while Areas have their own route definitions. This system ensures that each request is directed to the correct part of the application.

Controller – Logic Layer:

Controllers act as the “traffic directors” of the application. When a user visits a URL, the routing system maps that URL to a specific controller and action method. The controller receives the request, performs any necessary logic, interacts with the database through Models, and then selects a View to render the response.

The controller receives the users request (such as viewing rental history or creating a rental record)

For example:

```
// GET: Rent/RentalHistories
0 references
public ActionResult Index()
{
    return View(db.RentalHistories.ToList());
}
```

The index() controller method that runs when a user goes to the RentalHistories page, it returns a list of rental histories to the view.

The controller contains the logic to:

- Display lists of items
- Show details
- Create new records
- Edit existing records
- Delete records
- Handle validation errors.

The controller uses the Models to fetch or update data.

Models – Data Layer

Models represent the data and business logic of the application. They define the structure of the database tables and contain validation rules, relationships, and any logic related to the data. The project uses **Entity Framework**, which automatically maps these Models to SQL database tables and manages database migrations.

The rent models represent the data structure stored in the database (like RentalHistory)

```
8
9
10 7 references
11  public class RentalHistory
12  {
13      [Key]
14      0 references
15      public int RentalHistoryId { get; set; }
16
17      0 references
18      public bool RentalDamaged { get; set; }
19
20      0 references
21      public string DamagesIncurred { get; set; }
22
23      0 references
24      public string Rental { get; set; }
25  }
```

The controller gets this data and passes it to the view

Views – UI Layer

Views are Razor templates that generate the HTML sent to the user's browser. They display data passed from the controller and use Razor syntax (@) to mix C# logic with HTML. Views are responsible for the user interface and presentation layer.

The view layer renders the HTML for the user. It displays data from the model, renders forms for creating/editing records, with HTML and c#.

For example, for the home page, the razor file combined with the foreach loop in the Index.cshtml (index view) displays every rental history record that the Index() method sent to it.

```
Git Repositor...RUDPT3Sorting)  RentalHistoriesController.cs  Global.asax.cs
▼ @model IEnumerable<TheatreCMS3.Areas.Rent.Models.RentalHistory>
```

```

@foreach (var item in Model) {
    <tr>
        <td>
            @Html.DisplayFor(modelItem => item.RentalDamaged)
        </td>
        <td>
            @Html.DisplayFor(modelItem => item.DamagesIncurred)
        </td>
        <td>
            @Html.DisplayFor(modelItem => item.Rental)
        </td>
        <td>
            @Html.ActionLink("Edit", "Edit", new { id=item.RentalHistoryId }) |
            @Html.ActionLink("Details", "Details", new { id=item.RentalHistoryId }) |
            @Html.ActionLink("Delete", "Delete", new { id=item.RentalHistoryId })
        </td>
    </tr>
}

```

How I implemented the features

To build the Rental History feature, I followed the ASP.NET MVC workflow inside the Rent Area, using scaffolding, C#, Entity Framework, Razor, JavaScript, and custom UI design.

1. Scaffolding the MVC CRUD Structure

I began by using ASP.NET MVC scaffolding, which automatically generated:

- A full `RentalHistoriesController` with all CRUD action methods
- Strongly-typed Razor views for:
 - Create
 - Edit
 - Delete
 - Details
 - Index
- Form fields bound to the model using HTML helpers
- Validation and anti-forgery tokens

This provided the initial MVC structure, which I then expanded and customized.

2. Creating the Model (C# Classes, Properties, Data Annotations)

I created the `RentalHistory` model using core C# concepts:

- Classes to define the data structure
- Auto-implemented properties for each field
- Data annotations like `[Key]` to mark the primary key

This model represents one row in the `RentalHistory` SQL database table. It defines the data structure.

```

1  using System;
2  using System.Collections.Generic;
3  using System.ComponentModel.DataAnnotations;
4  using System.Linq;
5  using System.Web;
6
7  namespace TheatreCMS3.Areas.Rent.Models
8  {
9
10     public class RentalHistory
11     {
12         [Key]
13         public int RentalHistoryId { get; set; }
14
15         public bool RentalDamaged { get; set; }
16
17         public string DamagesIncurred { get; set; }
18
19         public string Rental { get; set; }
20     }
21 }

```

Entity framework then uses this class to create RentalHistories table, map each property to the column, track changes, and do all CRUD with data. It returns data as C# objects. This happens through DbSet<RentalHistory>:

```
public DbSet<RentalHistory> RentalHistories { get; set; }  
  
}
```

The rental histories controller interacts with the models constantly when fetching, adding, deleting, and editing records in this table.

Ex:

```
References  
public ActionResult Index()  
{  
    return View(db.RentalHistories.ToList());  
}
```

The razor views uses this model to display or edit data:

```

@model IEnumerable<TheatreCMS3.Areas.Rent.Models.RentalHistory>

@{
    ViewBag.Title = "Index";
}

<h2>Index</h2>

<p>
    @Html.ActionLink("Create New", "Create")
</p>

<!-- Select list for dropdown for sorting without reloading the page -->
<div style="display:flex; justify-content:flex-end; margin-bottom:10px;">
    <select id="sortSelect" class="form-select" style="width:220px;">
        <option value="nosort">No Extra Sorting</option>
        <option value="damaged">Damaged Rentals</option>
        <option value="undamaged">Undamaged Rentals</option>
        <option value="atoz">Rentals A - Z</option>
        <option value="ztoa">Rentals Z - A</option>
    </select>
</div>

<div class="list-group">

    <!--Parent element that holds all rows to order rental history rows(wrapped container)-->

    <div id="rentalList">
        <!-- added wrapper for JS sorting -->
        @foreach (var item in Model)
        {
            @* Showing all rental histories in a custom-designed Bootstrap layout *@
            @* One rental history entry *@
            <div class="list-group-item rental-row"
                data-id="@item.RentalHistoryId"
                data-name="@item.Rental ?? " " "
                data-damaged="@item.RentalDamaged.ToString().ToLower()">

                @* Main row *@
                <div>

```

And migrations reads the model and creates the SQL table:

```

CreateTable(
    "dbo.RentalHistories",
    c => new
    {
        RentalHistoryId = c.Int(nullable: false, identity: true),
        RentalDamaged = c.Boolean(nullable: false),
        DamagesIncurred = c.String(),
        Rental = c.String(),
    })
    .PrimaryKey(t => t.RentalHistoryId);

```

3. Database Integration (DbContext, EF, Migrations)

To connect the model to the database, I:

- Added a `DbSet<RentalHistory>` to `ApplicationDbContext`

```
11 references
public class ApplicationDbContext : IdentityDbContext<ApplicationUser>
{
    public DbSet<RentalHistory> RentalHistories { get; set; }
}
```

- Created an Entity Framework migration to generate the RentalHistories table
- Used EF features:
 - Change tracking
 - Code-first schema generation
 - Migration classes with Up() and Down() methods

This created the full backend storage layer.

4. Implementing the Controller (C# Methods, Model Binding, EF Operations)

The controller handles all logic for the feature. I used C# concepts such as:

- Action methods returning ActionResult
- Method overloading (GET vs POST actions)
- Model binding to map form fields to C# objects
- LINQ to query the database
- Entity Framework operations:
 - .ToList()
 - .Find(id)
 - .Add()
 - .Remove()
 - .SaveChanges()
 - db.Entry(entity).State = EntityState.Modified
- Validation using ModelState.IsValid
- Error handling for null or invalid IDs

This created the complete logic layer for CRUD operations.

Lets go through an example:

When I scaffolded, the tool automatically created the Index(), Details() and all other CRUD actions, including razor views. The actions are all automatically tied to their corresponding views. So controller action named Create() looks for view named Create.cshtml inside the folder matching the controller name. This is an automatic process.

```
// GET: Rent/RentalHistories/Create
0 references
public ActionResult Create()
{
    return View();
}
```

For action result Create(), this method is run my MVC when the user clicks Create New Rental History. It returns the empty Create.cshtml form. The post Create() actually takes that form and saves it.

```
// POST: Rent/RentalHistories/Create
// To protect from overposting attacks, enable the specific properties you want to bind to, for
// more details see https://go.microsoft.com/fwlink/?LinkId=317598.
[HttpPost]
[ValidateAntiForgeryToken]
0 references
public ActionResult Create([Bind(Include = "RentalHistoryId,RentalDamaged,DamagesIncurred,Rental")] RentalHistory rentalHistory)
{
    if (ModelState.IsValid)
    {
        db.RentalHistories.Add(rentalHistory);
        db.SaveChanges();
        return RedirectToAction("Index");
    }

    return View(rentalHistory);
}
```

When user presses submit, MVC takes the form fields and creates a RentalHistory object. (model binding) The modelstate.is valid portion checks to make sure the data is valid. Then db.RentalHistories.Add(rentalHistory) line adds the new record into the rental histories table, saves, redirects to index. Db.RentalHistories is the rental histories table in the DB, and comes from:

```
7 references
public DbSet<RentalHistory> RentalHistories { get; set; }
```

In identity models.

The same general concept for the rest of the action methods.

5. Customizing the Views (Razor, HTML Helpers, C# Expressions)

When I scaffolded the controller and views, VS automatically generated all of the cshtml files. In index.html, MVC creates a default HTML table. I replaced this with my own custom code.

Bootstrap list-group layout

I removed the generic <table> layout and, built a Bootstrap list-group interface that gave me more control over styling.

I started by creating a wrapper that holds all rental history rows where I first loop through each RentalHistory item using Razor. For each entry (data-id name etc., I render a custom row using Bootstrap):

```
<div class="list-group">

    <!--Parent element that holds all rows to order rental history rows(wrapped container)-->

    <div id="rentalList">
        <!-- added wrapper for JS sorting -->
        @foreach (var item in Model)
        {
            @* Showing all rental histories in a custom-designed Bootstrap layout *@
            @* One rental history entry *@
            <div class="list-group-item rental-row"
                data-id="@item.RentalHistoryId"
                data-name="@item.Rental ?? " " "
                data-damaged="@item.RentalDamaged.ToString().ToLower()">
```

I added data attributes here because I knew I would need them later for JavaScript sorting.

Flexbox row structure

Inside each row, I used Flexbox to create a three-column layout-

```
<div class="d-flex align-items-center rental-columns">
```

Font Awesome icons

- Green check for undamaged
- Red X for damaged

I used Razor if statements (Conditional c# logic) to show a green check or red X:

```

@*Main row*@
<div>

  <div class="d-flex align-items-center rental-columns">

    @* Column 1- Icon Column shows green check mark, or red x *@
    <div class="col-icon">
      @if (item.RentalDamaged)
      {
        <i class="fa-solid fa-xmark" style="color:#d9534f; font-size:1.5rem;"></i>
      }
      else
      {
        <i class="fa-solid fa-check" style="color:#5cb85c; font-size:1.5rem;"></i>
      }
    </div>

```

Badge styling for rental names, this includes conditional logic to show muted or dark text.

```

@* Column 2- Badge column shows the rental name badge *@
<div class="col-badge">
  <span class="badge fs-6" style="background-color:#3c3c3c; color:#f8f9fa;">
    @item.Rental
  </span>
</div>

@* Column 3- Damages column shows the damages incurred, gray out text if no damage. *@
<div class="col-damages">
  @*When a rental is not damages make the text dark. CSS makes the text default to white
  <span class="damage-text @((item.RentalDamaged ? "cms-text-dark" : "text-muted"))">
    @item.DamagesIncurred
  </span>
</div>

```

I added a n elipses that only appears when the user hovers over a row and a drop down menu when you click:

```

@*Elipses that show when you hover rows*@
<div class="action-menu dropdown">
  <span data-toggle="dropdown"
    aria-expanded="false"
    style="font-size: 30px; color: black; cursor: pointer;">
    ...
  </span>

  <ul class="dropdown-menu dropdown-menu-end">
    <li>@Html.ActionLink("Edit", "Edit", new { id = item.RentalHistoryId })</li>
    <li>@Html.ActionLink("Details", "Details", new { id = item.RentalHistoryId })</li>
    <li>@Html.ActionLink("Delete", "Delete", new { id = item.RentalHistoryId })</li>
  </ul>
</div>

```

Javascript:

To make the list sortable without reloading the page, I combined C# and JavaScript.

First, I added data attributes to each row in one of the previous steps:

```
<div class="list-group-item rental-row"
    data-id="@item.RentalHistoryId"
    data-name="@item.Rental ?? " " "
    data-damaged="@item.RentalDamaged.ToString().ToLower()">
```

These attributes let JavaScript read the values directly from the DOM.

Then I wrote a sorting script that listens for changes in the dropdown:

```
select.addEventListener("change", function () {
    var sortOption = select.value;
```

I grabbed all the rows and converted them into an array

```
//gettin all of the rows to sort
var rows = container.getElementsByClassName("rental-row");

// we need to convert the HTML collection to an array so we
//are able to .sort() method
var rentalRows = [];
for (var i = 0; i < rows.length; i++) {
    rentalRows.push(rows[i]);
}
```

Wrote the sorting logic:

```

var sortedRows;

//sorting logic
if (sortOption === "damaged") {
    sortedRows = rentalRows.sort(function (a, b) {
        return b.dataset.damaged.localeCompare(a.dataset.damaged);
    });
}
else if (sortOption === "undamaged") {
    sortedRows = rentalRows.sort(function (a, b) {
        return a.dataset.damaged.localeCompare(b.dataset.damaged);
    });
}
else if (sortOption === "atoz") {
    sortedRows = rentalRows.sort(function (a, b) {
        return a.dataset.name.localeCompare(b.dataset.name);
    });
}
else if (sortOption === "ztoa") {
    sortedRows = rentalRows.sort(function (a, b) {
        return b.dataset.name.localeCompare(a.dataset.name);
    });
}
else {
    // By default we are going to sort with the higher
    //id's first since they are the newest added entries.
    sortedRows = rentalRows.sort(function (a, b) {
        return b.dataset.id - a.dataset.id;
    });
}

```

Re-appended the sorted rows

```

// Update list without reloading the page
for (var j = 0; j < sortedRows.length; j++) {
    container.appendChild(sortedRows[j]);
}
});

```

This creates a fast and dynamic UI with both C# and JavaScript.

Integrating the Feature Into the Rent Area (Routing + MVC Structure)

Because the TheatreCMS3 project uses MVC Areas, I built my entire Rental History feature inside the Rent Area.

Areas/Rent/Models

Areas/Rent/Views

Areas/Rent/Controllers

To make this work, I had to understand and configure Area registration, routing, and folder structure.

This required:

- **Area registration**
- **Area-specific routing**
- **Matching controller and view names**
- **Using the Rent Area layout**

Every Area in MVC has its own registration file. I declared the area name “rent”, set up the route pattern for all controllers inside area, ensured MVS knows how to reach my controllers and views so the URLs look like this:

/Rent/RentalHistories/Index

/Rent/RentalHistories/Create

Inside the Rent Area, I created my controller here:

Areas/Rent/Controllers/RentalHistoriesController.cs

because it lives inside an Area, MVC automatically uses the Rent routing rules.

Example:

```
namespace TheatreCMS3.Areas.Rent.Controllers
{
    0 references
    public class RentalHistoriesController : Controller
    {
        private ApplicationDbContext db = new ApplicationDbContext();

        // GET: Rent/RentalHistories
        0 references
        public ActionResult Index()
        {
            return View(db.RentalHistories.ToList());
        }
    }
}
```